

Diseño de Algoritmos

ISIS2112

Complejidad computacional

Daniel Barrero

March 27, 2026

Computational complexity



Computational complexity



- ▶ We've used notation such as O and Ω to describe the worst-case complexity of *algorithms*.

Computational complexity



- ▶ We've used notation such as O and Ω to describe the worst-case complexity of *algorithms*.
- ▶ We must now turn to the complexity of *computational problems*.

Computational complexity — *Church-Turing thesis and models of computation*

Models of computation

- ▶ Turing machines ✓ (ISIS1106)
- ▶ RAM instructions ✓ (Every other course)

Computational complexity — *Church-Turing thesis and models of computation*

Models of computation

- ▶ Turing machines ✓ (ISIS1106)
- ▶ RAM instructions ✓ (Every other course)
- ▶ Lambda calculus
- ▶ Arithmetic circuits
- ▶ Boolean circuits

Computational complexity — *Church-Turing thesis and models of computation*

Models of computation

- ▶ Turing machines ✓ (ISIS1106)
- ▶ RAM instructions ✓ (Every other course)
- ▶ Lambda calculus
- ▶ Arithmetic circuits
- ▶ Boolean circuits

The **Church-Turing thesis** says that any two of these models are equivalent.

RAM instructions model of computation

- ▶ Our discussion of computational complexity and intractability will be in terms of the *RAM* model of computation.

RAM instructions model of computation

- ▶ Our discussion of computational complexity and intractability will be in terms of the *RAM* model of computation.
- ▶ We will use a **fixed programming language with types**, very Java-like.

RAM instructions model of computation

- ▶ Our discussion of computational complexity and intractability will be in terms of the *RAM* model of computation.
- ▶ We will use a **fixed programming language with types**, very Java-like.

We may nonetheless occasionally discuss some statements in terms of Turing machines

Decision problems

Definition

Let $\hat{X}$ be a type of our programming language and let $X \subset \hat{X}$. The *decision problem* defined by X consists of assigning **true** or **false** to the expression $x \in X$ for all $x \in \hat{X}$.

Decision problems

Definition

Let $\hat{X}$ be a type of our programming language and let $X \subset \hat{X}$. The *decision problem* defined by X consists of assigning **true** or **false** to the expression $x \in X$ for all $x \in \hat{X}$.

Example

1. $\hat{X} = \mathbf{Z}$, $X = \text{Primes}$.

Decision problems

Definition

Let $\hat{X}$ be a type of our programming language and let $X \subset \hat{X}$. The *decision problem* defined by X consists of assigning **true** or **false** to the expression $x \in X$ for all $x \in \hat{X}$.

Example

1. $\hat{X} = \mathbf{Z}$, $X = \text{Primes}$.
2. $\hat{X} = \text{Graph}$, $X = \text{connGraph}$. Namely, the problem of deciding whether or not a graph is connected.

Decision problems

Definition

Let $\hat{X}$ be a type of our programming language and let $X \subset \hat{X}$. The *decision problem* defined by X consists of assigning **true** or **false** to the expression $x \in X$ for all $x \in \hat{X}$.

Example

1. $\hat{X} = \mathbf{Z}$, $X = \text{Primes}$.
2. $\hat{X} = \text{Graph}$, $X = \text{connGraph}$. Namely, the problem of deciding whether or not a graph is connected.
3. $\hat{X} = \text{String}$, $X = \text{Python programs that stop}$.

Decision problems

Definition

Let $\hat{X}$ be a type of our programming language and let $X \subset \hat{X}$. The *decision problem* defined by X consists of assigning **true** or **false** to the expression $x \in X$ for all $x \in \hat{X}$.

Example

1. $\hat{X} = \mathbf{Z}$, $X = \text{Primes}$.
2. $\hat{X} = \text{Graph}$, $X = \text{connGraph}$. Namely, the problem of deciding whether or not a graph is connected.
3. $\hat{X} = \text{String}$, $X = \text{Python programs that stop}$.

Notation. We will use $X \subset \hat{X}$ to denote the decision problem defined by X .

Decision problems - computability

Definition

The decision problem $X \subset \hat{X}$ is *computable* if there exists a program

boolean checkX($\hat{X}$ x)

such that $\text{checkX}(x) = \mathbf{true}$ if and only if $x \in X$.

Decision problems - computability

Definition

The decision problem $X \subset \hat{X}$ is *computable* if there exists a program

boolean checkX($\hat{X}$ x)

such that $\text{checkX}(x) = \mathbf{true}$ if and only if $x \in X$.

Example

Examples 1 and 2 from the previous slide are computable, example 3 is not.

Computable decision problems - Complexity classes

Definition

The computable decision problem $X \subset \hat{X}$ is *of class P* or is said to have *polynomial time complexity* if there exists $k \in \mathbf{Z}^+$ such that

$$\text{check}X \in O(n^k),$$

where n is the size of the input $x \in \hat{X}$.

Computable decision problems - Complexity classes

Definition

The computable decision problem $X \subset \hat{X}$ is of class P or is said to have *polynomial time complexity* if there exists $k \in \mathbf{Z}^+$ such that

$$\text{checkX} \in O(n^k),$$

where n is the size of the input $x \in \hat{X}$.

Example

1. The problem $\text{Primes} \subset \mathbf{Z} \in P$ by taking $\text{checkX} = \text{isPrime}$ from the next slide.
2. The problem $\text{connGraph} \subset \text{Graph} \in P$ by taking $\text{checkX} = \text{dfs}$.

Computable decision problems - Complexity classes

This primality test is based on Erathosthenes' theorem.

```
boolean isPrime(int n) {  
    if (n <= 1) {  
        return false;  
    }  
    int r= (int) Math.sqrt(Math.abs(n));  
    for (int a= 2; a <= r; a++) {  
        if (n % a == 0) {  
            return false;  
        }  
    }  
    return true;  
}
```

Computable decision problems - Complexity classes

Definition

The computable decision problem $X \subset \hat{X}$ is of class *NP* or is said to have *non-deterministic polynomial time complexity* if there exist a type Q , a decision problem $Z \subset \hat{X} \times Q$, and a polynomial $p(n)$ satisfying the following conditions:

1. $x \in X$ if and only if there exists $q \in Q$ such that $(x, q) \in Z$.
2. For all $(x, q) \in Z$ the size of q is at most $p(n)$ where n is the size of x .
3. checkZ is of class *P*.

The decision problem $Z \subset \hat{X} \times Q$ is called a *proof system* for $X \subset \hat{X}$.

NP problems

Example

The problem

$$\overline{\text{Primes}} \subset \mathbf{Z}$$

of whether or not a number is composite admits the proof system

$$Z = \{(n, a) : 1 < |a| < |n|, \ n \bmod a = 0\}$$

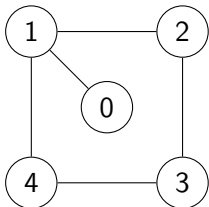
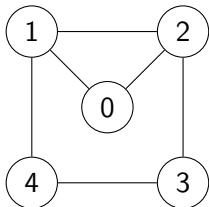
and $Z \subset \mathbf{Z} \times \mathbf{Z} \in P$ since it consists of dividing n by a .

Question

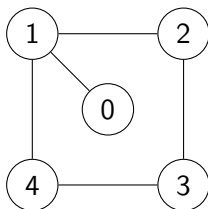
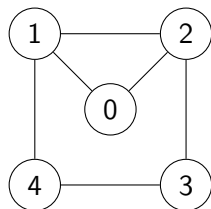
The above argument about the proof system $Z \subset \mathbf{Z} \times \mathbf{Z}$ only checks condition 3. Why does it also satisfy conditions 1 and 2?

Definition

A graph is said to be *Hamiltonian* if it contains a simple cycle.



NP problems - HamD



Example

The problem $\text{Ham} \subset \text{Graph}$ admits the proof system

$$Z = \{(g, \mathbf{u}) : B(g, \mathbf{u})\} \subset \text{Graph} \times \mathbf{Z}[]$$

where $B(g, \mathbf{u})$ is **true** if and only if $\mathbf{u} = [u_0, \dots, u_{n-1}]$ is a sequence of nodes in g such that $\{u_i, u_{(i+1) \bmod n}\} \in A_g$ for $i = 0, \dots, n-1$.

Quiz - 15 min



Quiz - 15 min



Demuestre que Z satisface las propiedades de un sistema de pruebas para HamD.

NP problems - HamD

$Z \subset \text{Graph} \times \mathbf{Z}[]$ from the previous slide is a proof system for $\text{Ham} \subset \text{Graph}$ because

1. Simple cycles on a graph can be represented with valid non-repeating sequences of nodes.
2. For each $(g, \mathbf{u}) \in Z$ we have $|\mathbf{u}| \in O(|g|)$.
3. checkZ is $O(n)$ because it scans $\mathbf{u}$ to check if it corresponds to an edge sequence that defines a cycle.

Notation. We use HamD , read as “*Hamiltonian decision*”, to denote the decision problem $\text{Ham} \subset \text{Graph}$.

Definition

Let $G = (N, A)$ be an undirected graph. A *cover* of G is a subset $N' \subset N$ such that $a \cap N' \neq \emptyset$ for all $a \in A$.

NP problems - CovD

Definition

Let

$$\hat{X} = \text{Graph} \times \mathbf{Z} \text{ and let}$$
$$X = \{(g, k) \mid g \text{ has a cover of size } k\}.$$

We then call $X \subset \hat{X}$ the *cover decision problem* and denote it by CovD.

It is of class *NP* because it admits a proof system $Z \subset \hat{X} \times \mathbf{Z}[]$ given by

$$Z = \{((g, k), \mathbf{u}) \mid \mathbf{u} \text{ defines a cover of } g \text{ of size } k\}$$

Reductions

Definition

We say that the problem $X \subset \hat{X}$ is *polynomial time reducible* to the problem $Y \subset \hat{Y}$ if there exists an algorithm

$$\hat{Y} \text{ translate}(\hat{X} \ x)$$

such that

1. $\tau_{\text{translate}} \in O(n^k)$ for some $k \in \mathbf{Z}^+$ where $n = |x|$.
2. $X = \text{translate}^{-1}(Y)$.

We will denote this by

$$X \subset \hat{X} \leq_P Y \subset \hat{Y}$$

and simply say *reducible* instead of “polynomial time reducible”.

Reductions

Theorem

1. $\text{CovD} \leq_p \text{HamD}$ (*hard*).

Reductions

Theorem

1. $\text{CovD} \leq_p \text{HamD}$ (*hard*).
2. $\text{HamD} \leq_p \text{TSPD}$ (*easy*).